Python Object-Oriented Programming (OOP)

Concept Overview Table

Concept	Keywords	Purpose
Class & Object	`class`, `__init__`, `self`	Encapsulate data and methods
Encapsulation	`_private`, `__private`, `@property`, `@setter`	Data hiding, validation
Constructor	`__init__()`, method overloading	Initialize objects
List of Objects	`[Class() for _ in range(n)]`	Store multiple objects in list
Inheritance	`class Child(Parent):`	Code reuse from base class
Abstraction	`ABC`, `@abstractmethod`	Hide implementation, show functionality
Multiple Inheritance	`class Child(Parent1, Parent2):`	Inherit from multiple classes

---

1. Class & Object

Concept:

- **Class**: A user-defined blueprint or prototype from which objects are created.
- **Object**: A real-world entity created from a class.

Example: Rectangle Class

```
# Class & Object in Python
class Rectangle:
    def __init__(self):
        self.length = 0
        self.width = 0

    def get_details(self, x, y):
        self.length = x
        self.width = y

    def area(self):
        return self.length * self.width

# Main execution
if __name__ == "__main__":
    # Creating first object
    o1 = Rectangle()
    o1.length = 10
    o1.width = 20
    print(f"Area of Rectangle: {o1.area()}")

    # Creating second object
```

```
    o2 = Rectangle()
    o2.get_details(20, 30)
    print(f"Area of Rectangle: {o2.area()}")
```

Output:

```
Area of Rectangle: 200
Area of Rectangle: 600
```

---

2. Encapsulation (Data Hiding using Properties)

Concept:

- Binding variables and methods together using private variables and property decorators.
- Ensures controlled access to class fields.

Example Without Encapsulation (Error Prone):

```
# Without proper encapsulation
class ShapeRectangle:
    def __init__(self):
        self.length = 0
        self.width = 0

    def area(self):
        return self.length * self.width

# This allows invalid values
obj = ShapeRectangle()
obj.length = -10  # This should not be allowed!
obj.width = 20
print(f"Area: {obj.area()}")  # Results in negative area
```

Example With Proper Encapsulation:

```
# Data Hiding using Properties in Python
class ShapeRectangle:
    def __init__(self):
        self._length = 0  # Protected attribute
        self._width = 0

    # Getter methods using @property
    @property
    def length(self):
        return self._length

    @property
    def width(self):
        return self._width

    # Setter methods with validation
    @length.setter
    def length(self, value):
        if value > 0:
```

```
            self._length = value
        else:
            self._length = 0
            print("Length cannot be negative or zero!")

    @width.setter
    def width(self, value):
        if value > 0:
            self._width = value
        else:
            self._width = 0
            print("Width cannot be negative or zero!")

    def area(self):
        return self._length * self._width

# Usage
if __name__ == "__main__":
    o1 = ShapeRectangle()
    o1.length = -10  # Will trigger validation
    o1.width = 20
    print(f"Length: {o1.length}")
    print(f"Width: {o1.width}")
    print(f"Area of Rectangle: {o1.area()}")
```

Output:

```
Length cannot be negative or zero!
Length: 0
Width: 20
Area of Rectangle: 0
```

---

3. Constructor & Constructor Overloading

Concept:

- A constructor (`__init__`) initializes an object.
- Python doesn't have traditional overloading, but we can simulate it using default parameters or class methods.

Basic Constructor Example:

```
# Constructor in Python
class RectangleShape:
    def __init__(self):
        print("Constructor Called")
        self.length = 2
        self.width = 10

    def area(self):
        return self.length * self.width

# Usage
if __name__ == "__main__":
    o1 = RectangleShape()
    print(f"Area of Rectangle: {o1.area()}")
```

Output:

```
Constructor Called
Area of Rectangle: 20
```

Constructor Overloading Simulation:

```
# Parameterized Constructor & Constructor "Overloading"
class Box:
    def __init__(self, length=2, breadth=5):
        """Default constructor with default parameters"""
        self.length = length
        self.breadth = breadth

    @classmethod
    def square_box(cls, side):
        """Alternative constructor for square box"""
        return cls(side, side)

    @classmethod
    def from_dimensions(cls, length, breadth):
        """Alternative constructor with explicit dimensions"""
        return cls(length, breadth)

    def area(self):
        return self.length * self.breadth

# Usage
if __name__ == "__main__":
    # Default constructor
    o = Box()
    print(f"Area: {o.area()}")

    # Parameterized constructor
    o1 = Box(3, 6)
    print(f"Area: {o1.area()}")

    # Square box using class method
    o2 = Box.square_box(3)
    print(f"Area: {o2.area()}")
```

Output:

```
Area: 10
Area: 18
Area: 9
```

---

4. List of Objects

Concept:

- Using lists to store multiple objects.

Example:

```
# List of objects
class Student:
    def __init__(self, roll_no, name):
        self.roll_no = roll_no
        self.name = name

    def print_details(self):
        print(f"Name: {self.name}")
        print(f"Roll No: {self.roll_no}")
        print("." * 20)

# Individual object creation
if __name__ == "__main__":
    o = Student(525111, "Narayana")
    o.print_details()

    o1 = Student(525621, "Subbu")
    o1.print_details()
```

Array/List of Objects:

```
# List of objects - Multiple students
class Student:
    def __init__(self, roll_no, name):
        self.roll_no = roll_no
        self.name = name

    def print_details(self):
        print(f"Name: {self.name}")
        print(f"Roll No: {self.roll_no}")
        print("." * 20)

# Usage with list
if __name__ == "__main__":
    # Creating a list of student objects
    students = []

    # Adding students to the list
    students.append(Student(10, "Narayana"))
    students.append(Student(11, "Hare_Ram"))
    students.append(Student(12, "Raman"))
    students.append(Student(13, "Subbu"))
    students.append(Student(14, "Kovidha"))

    # Printing all student details
    for student in students:
        student.print_details()

    # Alternative: List comprehension method
    student_data = [(10, "Narayana"), (11, "Hare_Ram"), (12, "Raman"),
                    (13, "Subbu"), (14, "Kovidha")]

    students_list = [Student(roll, name) for roll, name in student_data]

    print("\nUsing list comprehension:")
    for student in students_list:
        student.print_details()
```

---

5. Inheritance

Single Inheritance

Concept: One class inherits from another.

```
# Single Inheritance in Python
class Father:
    def house(self):
        print("Have Own 2BHK House.")

class Son(Father):
    def car(self):
        print("Have Own Audi car.")

# Usage
if __name__ == "__main__":
    o = Son()
    o.car()     # Son's method
    o.house()   # Inherited from Father
```

Output:

```
Have Own Audi car.
Have Own 2BHK House.
```

Multilevel Inheritance

Concept: Inheritance in a chain.

```
# Multilevel Inheritance in Python
class GrandFather:
    def house(self):
        print("3 BHK House.")

class Father(GrandFather):
    def land(self):
        print("5 Acres of Land.")

class Son(Father):
    def car(self):
        print("Own Audi car.")

# Usage
if __name__ == "__main__":
    o = Son()
    o.car()     # Son's method
    o.house()   # Inherited from GrandFather
    o.land()    # Inherited from Father
```

Output:

```
Own Audi car.
3 BHK House.
5 Acres of Land.
```

Hierarchical Inheritance

Concept: One superclass, multiple subclasses.

```
# Hierarchical Inheritance in Python
class Shape:
    def __init__(self):
        self.length = 0
        self.breadth = 0
        self.radius = 0

class Rectangle(Shape):
    def __init__(self, length, breadth):
        super().__init__()
        self.length = length
        self.breadth = breadth

    def rectangle_area(self):
        return self.length * self.breadth

class Circle(Shape):
    def __init__(self, radius):
        super().__init__()
        self.radius = radius

    def circle_area(self):
        return 3.14 * (self.radius ** 2)

class Square(Shape):
    def __init__(self, length):
        super().__init__()
        self.length = length

    def square_area(self):
        return self.length ** 2

# Usage
if __name__ == "__main__":
    o1 = Rectangle(2, 5)
    print(f"Area of Rectangle: {o1.rectangle_area()}")

    o2 = Circle(5)
    print(f"Area of Circle: {o2.circle_area()}")

    o3 = Square(3)
    print(f"Area of Square: {o3.square_area()}")
```

Output:

```
Area of Rectangle: 10
Area of Circle: 78.5
Area of Square: 9
```

---

6. Abstraction using Abstract Base Class (ABC)

Concept:

- Abstract class: Cannot be instantiated, contains abstract methods (without implementation) and concrete methods (with implementation).
- Use `ABC` (Abstract Base Class) and `@abstractmethod` decorator.

```
from abc import ABC, abstractmethod

# Abstract class example
class Shape(ABC):
    @abstractmethod
    def draw(self):
        pass  # Abstract method - must be implemented by subclasses

    def message(self):
        print("Message from Shape")  # Concrete method

class RectangleShape(Shape):
    def draw(self):
        print("Draw Rectangle using length & breadth")

# Usage
if __name__ == "__main__":
    # shape = Shape()  # This would cause an error
    o = RectangleShape()
    o.draw()
    o.message()
```

Output:

```
Draw Rectangle using length & breadth
Message from Shape
```

Extended Abstract Class Example:

```
from abc import ABC, abstractmethod

# Extended example for Abstract class
class Mobile(ABC):
    def voice_call(self):
        print("You can Make Voice Call")

    @abstractmethod
    def camera(self):
        pass

    @abstractmethod
    def touch_display(self):
        pass

class Samsung(Mobile):
    def camera(self):
        print("64 Mega pixel Camera")

    def touch_display(self):
        print("5.5 inch Display")

    def finger_print(self):
        print("In the Display fast finger sensor")

class Nokia(Mobile):
```

```
    def camera(self):
        print("2 Mega pixel Camera")

    def touch_display(self):
        print("4.8 inch Display")

# Usage
if __name__ == "__main__":
    M32 = Samsung()
    M32.voice_call()
    M32.touch_display()
    M32.camera()
    M32.finger_print()

    print("." * 31)

    N1 = Nokia()
    N1.voice_call()
    N1.touch_display()
    N1.camera()
```

Output:

```
You can Make Voice Call
5.5 inch Display
64 Mega pixel Camera
In the Display fast finger sensor
...............................
You can Make Voice Call
4.8 inch Display
2 Mega pixel Camera
```

---

7. Multiple Inheritance with Interfaces

Concept:

- Python supports multiple inheritance directly.
- We can simulate interfaces using abstract base classes.

Simple Interface Example:

```
from abc import ABC, abstractmethod

# Interface simulation using ABC
class Animal(ABC):
    @abstractmethod
    def sound(self):
        pass

    @abstractmethod
    def sleep(self):
        pass

class Dog(Animal):
    def sound(self):
        print("The Dog sounds like: Woof")
```

```
    def sleep(self):
        print("Dog sleeping")

# Usage
if __name__ == "__main__":
    o = Dog()
    o.sound()
    o.sleep()
```

Output:

```
The Dog sounds like: Woof
Dog sleeping
```

Key Differences: Python vs Java OOP

Feature	Python	Java
Constructor	`__init__(self)`	`ClassName()`
Access Modifiers	`_protected`, `__private` (convention)	`private`, `protected`, `public`
Multiple Inheritance	Supported directly	Via interfaces only
Method Overloading	Not directly supported	Supported
Abstract Classes	`from abc import ABC, abstractmethod`	`abstract class`, `abstract method`
Properties	`@property`, `@setter`	`get()`, `set()` methods
Static Methods	`@staticmethod`	`static` keyword
